

UsbCommander Client Application

Generated by Doxygen 1.16.1

1 Directory Hierarchy	1
1.1 Directories	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 Directory Documentation	7
4.1 src/main/java/com Directory Reference	7
4.2 src/test/java/com Directory Reference	7
4.3 src/main/java/com/usbcommander/config Directory Reference	7
4.4 src/main/java/com/usbcommander/config/contract Directory Reference	8
4.5 src/main/java/com/usbcommander/managers/contract Directory Reference	8
4.6 src/main/java/com/usbcommander/services/contract Directory Reference	8
4.7 src/main/java/com/usbcommander/dto Directory Reference	8
4.8 src/main/java/com/usbcommander/enums Directory Reference	8
4.9 src/main/java/com/usbcommander/errors Directory Reference	8
4.10 src/main/java Directory Reference	8
4.11 src/test/java Directory Reference	8
4.12 src/main Directory Reference	8
4.13 src/main/java/com/usbcommander/managers Directory Reference	8
4.14 src/main/java/com/usbcommander/services Directory Reference	9
4.15 src Directory Reference	9
4.16 src/test Directory Reference	9
4.17 src/main/java/com/usbcommander Directory Reference	9
4.18 src/test/java/com/usbcommander Directory Reference	9
5 Namespace Documentation	11
5.1 Package com.usbcommander	11
5.2 Package com.usbcommander.config	11
5.3 Package com.usbcommander.config.contract	11
5.4 Package com.usbcommander.dto	12
5.5 Package com.usbcommander.enums	12
5.6 Package com.usbcommander.errors	12
5.7 Package com.usbcommander.managers	12
5.8 Package com.usbcommander.managers.contract	12
5.9 Package com.usbcommander.services	12
5.10 Package com.usbcommander.services.contract	13
6 Class Documentation	15
6.1 com.usbcommander.App Class Reference	15
6.1.1 Member Function Documentation	15

6.1.1.1 main()	15
6.2 com.usbcommander.AppConst Class Reference	15
6.2.1 Member Data Documentation	15
6.2.1.1 MAIN_LOCATION	15
6.3 com.usbcommander.AppTest Class Reference	16
6.3.1 Detailed Description	16
6.3.2 Member Function Documentation	16
6.3.2.1 shouldAnswerWithTrue()	16
6.4 com.usbcommander.services.contract.CommanderService Class Reference	16
6.4.1 Detailed Description	16
6.4.2 Constructor & Destructor Documentation	16
6.4.2.1 CommanderService()	16
6.4.3 Member Function Documentation	17
6.4.3.1 run()	17
6.4.3.2 start()	17
6.4.3.3 stopService()	17
6.4.4 Member Data Documentation	17
6.4.4.1 running	17
6.5 com.usbcommander.dto.ConfigDTO Class Reference	17
6.5.1 Detailed Description	18
6.5.2 Constructor & Destructor Documentation	18
6.5.2.1 ConfigDTO()	18
6.5.3 Member Function Documentation	18
6.5.3.1 getAllowedTime()	18
6.5.3.2 getFrecuency()	18
6.5.3.3 isAllow()	18
6.5.3.4 setAllow()	18
6.5.3.5 setAllowedTime()	18
6.5.3.6 setFrecuency()	18
6.5.4 Member Data Documentation	19
6.5.4.1 allow	19
6.5.4.2 allowedTime	19
6.5.4.3 frecuency	19
6.6 com.usbcommander.AppConst.ConfigReferences Class Reference	19
6.6.1 Member Data Documentation	19
6.6.1.1 CONFIG_LOCATION	19
6.6.1.2 IP_ENTRY	19
6.6.1.3 LOG_ENTRY	19
6.6.1.4 MIN_FRECUENNCY	19
6.6.1.5 PORT_ENTRY	20
6.6.1.6 USB_ALLOW_ENTRY	20
6.7 com.usbcommander.AppConst.ErrorMessages Class Reference	20

6.7.1 Member Data Documentation	20
6.7.1.1 CONNECTION_ERROR_MESSAGE	20
6.7.1.2 ERROR_DELETE	20
6.7.1.3 JACKSON_ERROR_TEXT	20
6.7.1.4 LOG_GENERATION_ERROR	20
6.7.1.5 SERVICE_NOT_RUNNING	21
6.7.1.6 WEIRDVALUE_ERROR	21
6.8 com.usbcommander.AppConst.EventLogReferences Class Reference	21
6.8.1 Member Data Documentation	21
6.8.1.1 ENTRY_NAME	21
6.8.1.2 ENTRY_ROUTE	21
6.8.1.3 ERROR_CODE	21
6.8.1.4 ERROR_CONNECTION_CODE	21
6.8.1.5 INCOHERENT_VALUE	22
6.8.1.6 INFO_CODE	22
6.8.1.7 MEMORY_CONNECTED	22
6.8.1.8 REGISTRY_MODIFICATION	22
6.8.1.9 UNAUTHORIZED_CONFIGURATION_MODIFICATION	22
6.9 com.usbcommander.config.contract.IMachineConfig Class Reference	22
6.9.1 Detailed Description	22
6.9.2 Member Function Documentation	23
6.9.2.1 enableServerService()	23
6.9.2.2 getIp()	23
6.9.2.3 getLogFrequency()	23
6.9.2.4 getPort()	23
6.9.2.5 isUsbEnable()	24
6.9.2.6 saveConfig()	24
6.9.2.7 setLogFrequency()	24
6.9.2.8 setUsbEnable()	25
6.9.3 Member Data Documentation	25
6.9.3.1 instance	25
6.10 com.usbcommander.managers.contract.IStatusManager Class Reference	25
6.10.1 Detailed Description	25
6.10.2 Constructor & Destructor Documentation	26
6.10.2.1 IStatusManager()	26
6.10.3 Member Function Documentation	26
6.10.3.1 deleteHistory()	26
6.10.3.2 generateLog() [1/3]	26
6.10.3.3 generateLog() [2/3]	26
6.10.3.4 generateLog() [3/3]	27
6.10.3.5 getHistory()	27
6.10.4 Member Data Documentation	27

6.10.4.1 instance	27
6.11 com.usbcommander.managers.contract.IUsbMemoryManager Class Reference	27
6.11.1 Detailed Description	28
6.11.2 Constructor & Destructor Documentation	28
6.11.2.1 IUsbMemoryManager()	28
6.11.3 Member Function Documentation	28
6.11.3.1 disableAccess()	28
6.11.3.2 enableAccess()	28
6.11.3.3 getAccessValue()	28
6.11.3.4 getConnectedDrives()	29
6.11.3.5 isDriveConnected()	29
6.11.3.6 removeExternalDrives()	29
6.11.4 Member Data Documentation	29
6.11.4.1 instance	29
6.12 com.usbcommander.dto.LogDTO Class Reference	30
6.12.1 Detailed Description	30
6.12.2 Constructor & Destructor Documentation	30
6.12.2.1 LogDTO() [1/3]	30
6.12.2.2 LogDTO() [2/3]	30
6.12.2.3 LogDTO() [3/3]	31
6.12.3 Member Function Documentation	31
6.12.3.1 getCode()	31
6.12.3.2 getCreationDate()	31
6.12.3.3 getErrorMessage()	31
6.12.3.4 getUsbList()	31
6.12.3.5 getUsbValue()	31
6.12.3.6 isUsbAllowed()	31
6.12.3.7 setCode()	31
6.12.3.8 setCreationDate()	31
6.12.3.9 setErrorMessage()	32
6.12.3.10 setUsbAllowed()	32
6.12.3.11 setUsbList()	32
6.12.3.12 setUsbValue()	32
6.12.4 Member Data Documentation	32
6.12.4.1 code	32
6.12.4.2 creationDate	32
6.12.4.3 errorMessage	32
6.12.4.4 usbAllowed	32
6.12.4.5 usbList	32
6.12.4.6 usbValue	33
6.13 com.usbcommander.services.LogService Class Reference	33
6.13.1 Detailed Description	34

6.13.2 Constructor & Destructor Documentation	34
6.13.2.1 LogService()	34
6.13.3 Member Function Documentation	34
6.13.3.1 getInstance()	34
6.13.3.2 run()	34
6.13.3.3 sendLogs()	34
6.13.4 Member Data Documentation	34
6.13.4.1 instance	34
6.13.4.2 machineConfig	35
6.13.4.3 mapper	35
6.13.4.4 statusManager	35
6.14 com.usbcommander.enums.LogType Enum Reference	35
6.14.1 Detailed Description	35
6.14.2 Constructor & Destructor Documentation	36
6.14.2.1 LogType()	36
6.14.3 Member Function Documentation	36
6.14.3.1 getCode()	36
6.14.4 Member Data Documentation	36
6.14.4.1 code	36
6.14.4.2 CONFIG_MOD	36
6.14.4.3 CONNECTION	36
6.14.4.4 ERROR	36
6.14.4.5 INCOHERENT	36
6.14.4.6 INFO	36
6.14.4.7 MEMORY_CONN	36
6.14.4.8 REGISTRY_MOD	37
6.15 com.usbcommander.config.MachineConfig Class Reference	37
6.15.1 Detailed Description	37
6.15.2 Constructor & Destructor Documentation	38
6.15.2.1 MachineConfig() [1/2]	38
6.15.2.2 MachineConfig() [2/2]	38
6.15.3 Member Function Documentation	38
6.15.3.1 enableServerService()	38
6.15.3.2 getInstance()	38
6.15.3.3 getIp()	39
6.15.3.4 getLogFrequency()	39
6.15.3.5 getPort()	39
6.15.3.6 isUsbEnable()	39
6.15.3.7 saveConfig()	40
6.15.3.8 setLogFrequency()	40
6.15.3.9 setUsbEnable()	40
6.15.4 Member Data Documentation	40

6.15.4.1 ip	40
6.15.4.2 logFrequency	41
6.15.4.3 port	41
6.15.4.4 usbEnable	41
6.16 com.usbcommander.errors.NotConnectionException Class Reference	41
6.16.1 Detailed Description	41
6.16.2 Constructor & Destructor Documentation	41
6.16.2.1 NotConnectionException()	41
6.17 com.usbcommander.AppConst.RegistryReferences Class Reference	41
6.17.1 Member Data Documentation	42
6.17.1.1 DISABLE_VALUE	42
6.17.1.2 ENABLE_VALUE	42
6.17.1.3 USB_ENUM	42
6.17.1.4 USB_STOR	42
6.18 com.usbcommander.services.SecurityService Class Reference	42
6.18.1 Detailed Description	43
6.18.2 Constructor & Destructor Documentation	43
6.18.2.1 SecurityService()	43
6.18.3 Member Function Documentation	43
6.18.3.1 changeLogFrequency()	43
6.18.3.2 configMatchesMemory()	44
6.18.3.3 forceClose()	44
6.18.3.4 getInstance()	44
6.18.3.5 openAccess()	44
6.18.3.6 run()	45
6.18.3.7 setListenerOn()	45
6.18.3.8 startListener()	45
6.18.3.9 usbStorMatchesMemory()	46
6.18.4 Member Data Documentation	46
6.18.4.1 appConfigListener	46
6.18.4.2 grantedAccess	46
6.18.4.3 grantedFor	46
6.18.4.4 instance	46
6.18.4.5 machineConfig	46
6.18.4.6 statusManager	47
6.18.4.7 usbMemoryManager	47
6.18.4.8 usbStorListener	47
6.19 com.usbcommander.services.ServerConnectionService Class Reference	47
6.19.1 Detailed Description	48
6.19.2 Constructor & Destructor Documentation	48
6.19.2.1 ServerConnectionService()	48
6.19.3 Member Function Documentation	48

6.19.3.1 connectToServer()	48
6.19.3.2 getInstance()	49
6.19.3.3 listenerThread()	49
6.19.3.4 run()	49
6.19.3.5 sendMessage()	49
6.19.4 Member Data Documentation	50
6.19.4.1 CONN_ERROR	50
6.19.4.2 connected	50
6.19.4.3 instance	50
6.19.4.4 machineConfig	50
6.19.4.5 mapper	50
6.19.4.6 MAX_RETRY	51
6.19.4.7 MAX_RETRY_MSG	51
6.19.4.8 RECONNECT_TIMEOUT	51
6.19.4.9 SENDING_ERROR	51
6.19.4.10 SERVER_MSG_ERROR	51
6.19.4.11 socket	51
6.19.4.12 statusManager	51
6.20 com.usbcommander.errors.ServiceDisabledException Class Reference	52
6.20.1 Detailed Description	52
6.20.2 Constructor & Destructor Documentation	52
6.20.2.1 ServiceDisabledException()	52
6.21 com.usbcommander.managers.StatusManager Class Reference	52
6.21.1 Detailed Description	53
6.21.2 Constructor & Destructor Documentation	53
6.21.2.1 StatusManager()	53
6.21.3 Member Function Documentation	53
6.21.3.1 deleteHistory()	53
6.21.3.2 generateLog() [1/3]	53
6.21.3.3 generateLog() [2/3]	54
6.21.3.4 generateLog() [3/3]	54
6.21.3.5 getHistory()	54
6.21.3.6 getInstance()	55
6.21.3.7 writeEventLog()	55
6.21.4 Member Data Documentation	55
6.21.4.1 advapi32	55
6.21.4.2 machineConfig	55
6.21.4.3 mapper	55
6.21.4.4 usbMemoryManager	56
6.22 com.usbcommander.services.ThreadService Class Reference	56
6.22.1 Detailed Description	57
6.22.2 Constructor & Destructor Documentation	57

6.22.2.1 ThreadService()	57
6.22.3 Member Function Documentation	57
6.22.3.1 getInstance()	57
6.22.3.2 run()	57
6.22.4 Member Data Documentation	57
6.22.4.1 instance	57
6.22.4.2 machineConfig	57
6.22.4.3 statusManager	58
6.23 com.usbcommander.managers.UsbMemoryManager Class Reference	58
6.23.1 Detailed Description	59
6.23.2 Constructor & Destructor Documentation	59
6.23.2.1 UsbMemoryManager()	59
6.23.3 Member Function Documentation	59
6.23.3.1 disableAccess()	59
6.23.3.2 enableAccess()	59
6.23.3.3 getAccessValue()	59
6.23.3.4 getConnectedDrives()	60
6.23.3.5 getInstance()	60
6.23.3.6 isDriveConnected()	60
6.23.3.7 removeDrive()	60
6.23.3.8 removeExternalDrives()	61
6.23.4 Member Data Documentation	61
6.23.4.1 IOCTL_STORAGE_EJECT_MEDIA	61
Index	63

Chapter 1

Directory Hierarchy

1.1 Directories

com	7
usbcommander	9
config	7
contract	8
dto	8
enums	8
errors	8
managers	8
contract	8
services	9
contract	8
com	7
usbcommander	9
config	7
contract	8
contract	8
contract	8
contract	8
dto	8
enums	8
errors	8
java	8
com	7
usbcommander	9
config	7
contract	8
dto	8
enums	8
errors	8
managers	8
contract	8
services	9
contract	8
java	8
com	7
usbcommander	9

main	8
java	8
com	7
usbcommander	9
config	7
contract	8
dto	8
enums	8
errors	8
managers	8
contract	8
services	9
contract	8
managers	8
contract	8
services	9
contract	8
src	9
main	8
java	8
com	7
usbcommander	9
config	7
contract	8
dto	8
enums	8
errors	8
managers	8
contract	8
services	9
contract	8
test	9
java	8
com	7
usbcommander	9
test	9
java	8
com	7
usbcommander	9
usbcommander	9
config	7
contract	8
dto	8
enums	8
errors	8
managers	8
contract	8
services	9
contract	8
usbcommander	9

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

com.usbcommander.App	15
com.usbcommander.AppConst	15
com.usbcommander.AppTest	16
com.usbcommander.dto.ConfigDTO	17
com.usbcommander.AppConst.ConfigReferences	19
com.usbcommander.AppConst.ErrorMessages	20
com.usbcommander.AppConst.EventLogReferences	21
Exception	
com.usbcommander.errors.NotConnectionException	41
com.usbcommander.errors.ServiceDisabledException	52
com.usbcommander.config.contract.IMachineConfig	22
com.usbcommander.config.MachineConfig	37
com.usbcommander.managers.contract.IStatusManager	25
com.usbcommander.managers.StatusManager	52
com.usbcommander.managers.contract.IUsbMemoryManager	27
com.usbcommander.managers.UsbMemoryManager	58
com.usbcommander.dto.LogDTO	30
com.usbcommander.enums.LogType	35
com.usbcommander.AppConst.RegistryReferences	41
Thread	
com.usbcommander.services.contract.CommanderService	16
com.usbcommander.services.LogService	33
com.usbcommander.services.SecurityService	42
com.usbcommander.services.ServerConnectionService	47
com.usbcommander.services.ThreadService	56

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

com.usbcommander.App	15
com.usbcommander.AppConst	15
com.usbcommander.AppTest	16
com.usbcommander.services.contract.CommanderService	16
com.usbcommander.dto.ConfigDTO	17
com.usbcommander.AppConst.ConfigReferences	19
com.usbcommander.AppConst.ErrorMessages	20
com.usbcommander.AppConst.EventLogReferences	21
com.usbcommander.config.contract.IMachineConfig	22
com.usbcommander.managers.contract.IStatusManager	25
com.usbcommander.managers.contract.IUsbMemoryManager	27
com.usbcommander.dto.LogDTO	30
com.usbcommander.services.LogService	33
com.usbcommander.enums.LogType	35
com.usbcommander.config.MachineConfig	37
com.usbcommander.errors.NotConnectionException	41
com.usbcommander.AppConst.RegistryReferences	41
com.usbcommander.services.SecurityService	42
com.usbcommander.services.ServerConnectionService	47
com.usbcommander.errors.ServiceDisabledException	52
com.usbcommander.managers.StatusManager	52
com.usbcommander.services.ThreadService	56
com.usbcommander.managers.UsbMemoryManager	58

Chapter 4

Directory Documentation

4.1 `src/main/java/com` Directory Reference

Directories

- directory `usbcommander`

4.2 `src/test/java/com` Directory Reference

Directories

- directory `usbcommander`

4.3 `src/main/java/com/usbcommander/config` Directory Reference

Directories

- directory `contract`

4.4 `src/main/java/com/usbcommander/config/contract` Directory Reference

4.5 `src/main/java/com/usbcommander/managers/contract` Directory Reference

4.6 `src/main/java/com/usbcommander/services/contract` Directory Reference

4.7 `src/main/java/com/usbcommander/dto` Directory Reference

4.8 `src/main/java/com/usbcommander/enums` Directory Reference

4.9 `src/main/java/com/usbcommander/errors` Directory Reference

4.10 `src/main/java` Directory Reference

Directories

- directory `com`

4.11 `src/test/java` Directory Reference

Directories

- directory `com`

4.12 `src/main` Directory Reference

Directories

- directory `java`

4.13 `src/main/java/com/usbcommander/managers` Directory Reference

Directories

- directory `contract`

4.14 src/main/java/com/usbcommander/services Directory Reference

Directories

- directory **contract**

4.15 src Directory Reference

Directories

- directory **main**
- directory **test**

4.16 src/test Directory Reference

Directories

- directory **java**

4.17 src/main/java/com/usbcommander Directory Reference

Directories

- directory **config**
- directory **dto**
- directory **enums**
- directory **errors**
- directory **managers**
- directory **services**

4.18 src/test/java/com/usbcommander Directory Reference

Chapter 5

Namespace Documentation

5.1 Package com.usbcommander

Packages

- package **config**
- package **dto**
- package **enums**
- package **errors**
- package **managers**
- package **services**

Classes

- class **App**
- class **AppConst**
- class **AppTest**

5.2 Package com.usbcommander.config

Packages

- package **contract**

Classes

- class **MachineConfig**

5.3 Package com.usbcommander.config.contract

Classes

- class **IMachineConfig**

5.4 Package com.usbcommander.dto

Classes

- class **ConfigDTO**
- class **LogDTO**

5.5 Package com.usbcommander.enums

Classes

- enum **LogType**

5.6 Package com.usbcommander.errors

Classes

- class **NotConnectionException**
- class **ServiceDisabledException**

5.7 Package com.usbcommander.managers

Packages

- package **contract**

Classes

- class **StatusManager**
- class **UsbMemoryManager**

5.8 Package com.usbcommander.managers.contract

Classes

- class **IStatusManager**
- class **IUsbMemoryManager**

5.9 Package com.usbcommander.services

Packages

- package **contract**

Classes

- class **LogService**
- class **SecurityService**
- class **ServerConnectionService**
- class **ThreadService**

5.10 Package com.usbcommander.services.contract**Classes**

- class **CommanderService**

Chapter 6

Class Documentation

6.1 com.usbcommander.App Class Reference

Static Public Member Functions

- static void **main** (String[] args)

6.1.1 Member Function Documentation

6.1.1.1 main()

```
void com.usbcommander.App.main (  
    String[] args) [static]
```

6.2 com.usbcommander.AppConst Class Reference

Classes

- class **ConfigReferences**
- class **RegistryReferences**
- class **EventLogReferences**
- class **ErrorMessages**

Static Public Attributes

- static final HKEY **MAIN_LOCATION** = WinReg.HKEY_LOCAL_MACHINE

6.2.1 Member Data Documentation

6.2.1.1 MAIN_LOCATION

```
final HKEY com.usbcommander.AppConst.MAIN_LOCATION = WinReg.HKEY_LOCAL_MACHINE [static]
```

6.3 com.usbcommander.AppTest Class Reference

Public Member Functions

- void **shouldAnswerWithTrue** ()

6.3.1 Detailed Description

Unit test for simple **App** (p. 15).

6.3.2 Member Function Documentation

6.3.2.1 shouldAnswerWithTrue()

```
void com.usbcommander.AppTest.shouldAnswerWithTrue ()
```

Rigorous Test :-)

6.4 com.usbcommander.services.contract.CommanderService Class Reference

Public Member Functions

- abstract void **run** ()
- synchronized void **start** ()
- void **stopService** ()

Protected Member Functions

- **CommanderService** ()

Protected Attributes

- boolean **running**

6.4.1 Detailed Description

Clase abstracta usada como base para la creación de las clases "Servicio" que llevarán a cabo el control de todas las acciones de la aplicación

6.4.2 Constructor & Destructor Documentation

6.4.2.1 CommanderService()

```
com.usbcommander.services.contract.CommanderService.CommanderService () [protected]
```

6.4.3 Member Function Documentation

6.4.3.1 run()

```
abstract void com.usbcommander.services.contract.CommanderService.run () [abstract]
```

Reimplemented in **com.usbcommander.services.LogService** (p. 34), **com.usbcommander.services.SecurityService** (p. 45), **com.usbcommander.services.ServerConnectionService** (p. 49), and **com.usbcommander.services.ThreadService** (p. 57).

6.4.3.2 start()

```
synchronized void com.usbcommander.services.contract.CommanderService.start ()
```

6.4.3.3 stopService()

```
void com.usbcommander.services.contract.CommanderService.stopService ()
```

Método utilizado para detener el servicio correctamente

6.4.4 Member Data Documentation

6.4.4.1 running

```
boolean com.usbcommander.services.contract.CommanderService.running [protected]
```

Variable que almacena si el Hilo del servicio esta en ejecución o no

6.5 com.usbcommander.dto.ConfigDTO Class Reference

Public Member Functions

- **ConfigDTO** ()
- Boolean **isAllow** ()
- void **setAllow** (Boolean **allow**)
- Long **getFrequency** ()
- void **setFrequency** (Long **frequency**)
- Long **getAllowedTime** ()
- void **setAllowedTime** (Long **allowedTime**)

Private Attributes

- Boolean **allow**
- Long **frequency**
- Long **allowedTime**

6.5.1 Detailed Description

Clase usada únicamente como DTO (Data transfer object) para ser enviada y recibida en comunicaciones entre el cliente y el servidor en relación a la configuración de la aplicación cliente

6.5.2 Constructor & Destructor Documentation

6.5.2.1 ConfigDTO()

```
com.usbcommander.dto.ConfigDTO.ConfigDTO ()
```

6.5.3 Member Function Documentation

6.5.3.1 getAllowedTime()

```
Long com.usbcommander.dto.ConfigDTO.getAllowedTime ()
```

6.5.3.2 getFrecuency()

```
Long com.usbcommander.dto.ConfigDTO.getFrecuency ()
```

6.5.3.3 isAllow()

```
Boolean com.usbcommander.dto.ConfigDTO.isAllow ()
```

6.5.3.4 setAllow()

```
void com.usbcommander.dto.ConfigDTO.setAllow (  
    Boolean allow)
```

6.5.3.5 setAllowedTime()

```
void com.usbcommander.dto.ConfigDTO.setAllowedTime (  
    Long allowedTime)
```

6.5.3.6 setFrecuency()

```
void com.usbcommander.dto.ConfigDTO.setFrecuency (  
    Long frecuency)
```

6.5.4 Member Data Documentation

6.5.4.1 allow

Boolean com.usbcommander.dto.ConfigDTO.allow [private]

6.5.4.2 allowedTime

Long com.usbcommander.dto.ConfigDTO.allowedTime [private]

6.5.4.3 frequency

Long com.usbcommander.dto.ConfigDTO.frequency [private]

6.6 com.usbcommander.AppConst.ConfigReferences Class Reference

Static Public Attributes

- static final String **CONFIG_LOCATION** = "SOFTWARE\\UsbCmmdr"
- static final String **USB_ALLOW_ENTRY** = "allowUsb"
- static final String **LOG_ENTRY** = "logFrec"
- static final String **IP_ENTRY** = "ipdir"
- static final String **PORT_ENTRY** = "port"
- static final long **MIN_FRECUENNCY** = 300000

6.6.1 Member Data Documentation

6.6.1.1 CONFIG_LOCATION

final String com.usbcommander.AppConst.ConfigReferences.CONFIG_LOCATION = "SOFTWARE\\UsbCmmdr"
[static]

6.6.1.2 IP_ENTRY

final String com.usbcommander.AppConst.ConfigReferences.IP_ENTRY = "ipdir" [static]

6.6.1.3 LOG_ENTRY

final String com.usbcommander.AppConst.ConfigReferences.LOG_ENTRY = "logFrec" [static]

6.6.1.4 MIN_FRECUENNCY

final long com.usbcommander.AppConst.ConfigReferences.MIN_FRECUENNCY = 300000 [static]

6.6.1.5 PORT_ENTRY

```
final String com.usbcommander.AppConst.ConfigReferences.PORT_ENTRY = "port" [static]
```

6.6.1.6 USB_ALLOW_ENTRY

```
final String com.usbcommander.AppConst.ConfigReferences.USB_ALLOW_ENTRY = "allowUsb" [static]
```

6.7 com.usbcommander.AppConst.ErrorMessages Class Reference

Static Public Attributes

- static final String **SERVICE_NOT_RUNNING** = "Could not complete task as the service is not running. Error Info: "
- static final String **CONNECTION_ERROR_MESSAGE** = "Could not complete task as there is no connection established. Error Info: "
- static final String **JACKSON_ERROR_TEXT** = "Error while creating text from class. Error Info: "
- static final String **LOG_GENERATION_ERROR** = "Couldn't read machine information, log values couldn't be set. Error Info: "
- static final String **ERROR_DELETE** = "Error deleting event log history. Error Info: "
- static final String **WEIRDVALUE_ERROR** = "WeirdValue found while parsing logs. Error Info: "

6.7.1 Member Data Documentation

6.7.1.1 CONNECTION_ERROR_MESSAGE

```
final String com.usbcommander.AppConst.ErrorMessages.CONNECTION_ERROR_MESSAGE = "Could not complete task as there is no connection established. Error Info: " [static]
```

6.7.1.2 ERROR_DELETE

```
final String com.usbcommander.AppConst.ErrorMessages.ERROR_DELETE = "Error deleting event log history. Error Info: " [static]
```

6.7.1.3 JACKSON_ERROR_TEXT

```
final String com.usbcommander.AppConst.ErrorMessages.JACKSON_ERROR_TEXT = "Error while creating text from class. Error Info: " [static]
```

6.7.1.4 LOG_GENERATION_ERROR

```
final String com.usbcommander.AppConst.ErrorMessages.LOG_GENERATION_ERROR = "Couldn't read machine information, log values couldn't be set. Error Info: " [static]
```

6.7.1.5 SERVICE_NOT_RUNNING

```
final String com.usbcommander.AppConst.ErrorMessages.SERVICE_NOT_RUNNING = "Could not complete  
task as the service is not running. Error Info: " [static]
```

6.7.1.6 WEIRDVALUE_ERROR

```
final String com.usbcommander.AppConst.ErrorMessages.WEIRDVALUE_ERROR = "WeirdValue found  
while parsing logs. Error Info: " [static]
```

6.8 com.usbcommander.AppConst.EventLogReferences Class Reference

Static Public Attributes

- static final String **ENTRY_ROUTE** = "SYSTEM\\CurrentControlSet\\Services\\EventLog\\USBCommander↵
Log"
- static final String **ENTRY_NAME** = "UsbCommndr"
- static final int **INFO_CODE** = 1001
- static final int **UNAUTHORIZED_CONFIGURATION_MODIFICATION** = 1002
- static final int **REGISTRY_MODIFICATION** = 1003
- static final int **INCOHERENT_VALUE** = 1004
- static final int **MEMORY_CONNECTED** = 1005
- static final int **ERROR_CONNECTION_CODE** = 1006
- static final int **ERROR_CODE** = 1007

6.8.1 Member Data Documentation

6.8.1.1 ENTRY_NAME

```
final String com.usbcommander.AppConst.EventLogReferences.ENTRY_NAME = "UsbCommndr" [static]
```

6.8.1.2 ENTRY_ROUTE

```
final String com.usbcommander.AppConst.EventLogReferences.ENTRY_ROUTE = "SYSTEM\\Current↵  
ControlSet\\Services\\EventLog\\USBCommanderLog" [static]
```

6.8.1.3 ERROR_CODE

```
final int com.usbcommander.AppConst.EventLogReferences.ERROR_CODE = 1007 [static]
```

6.8.1.4 ERROR_CONNECTION_CODE

```
final int com.usbcommander.AppConst.EventLogReferences.ERROR_CONNECTION_CODE = 1006 [static]
```

6.8.1.5 INCOHERENT_VALUE

```
final int com.usbcommander.AppConst.EventLogReferences.INCOHERENT_VALUE = 1004 [static]
```

6.8.1.6 INFO_CODE

```
final int com.usbcommander.AppConst.EventLogReferences.INFO_CODE = 1001 [static]
```

6.8.1.7 MEMORY_CONNECTED

```
final int com.usbcommander.AppConst.EventLogReferences.MEMORY_CONNECTED = 1005 [static]
```

6.8.1.8 REGISTRY_MODIFICATION

```
final int com.usbcommander.AppConst.EventLogReferences.REGISTRY_MODIFICATION = 1003 [static]
```

6.8.1.9 UNAUTHORIZED_CONFIGURATION_MODIFICATION

```
final int com.usbcommander.AppConst.EventLogReferences.UNAUTHORIZED_CONFIGURATION_MODIFICATION  
= 1002 [static]
```

6.9 com.usbcommander.config.contract.IMachineConfig Class Reference

Public Member Functions

- abstract boolean **saveConfig** ()
- abstract boolean **isUsbEnable** ()
- abstract long **getLogFrequency** ()
- abstract int **getPort** ()
- abstract String **getIp** ()
- abstract void **setUsbEnable** (boolean usbEnable)
- abstract void **setLogFrequency** (long logFrequency)
- abstract void **enableServerService** ()

Static Protected Attributes

- static **IMachineConfig** instance

6.9.1 Detailed Description

Clase abstracta usada como base para configurar métodos y controlar valores en relación a la configuración de la aplicación en la máquina

6.9.2 Member Function Documentation

6.9.2.1 enableServerService()

```
abstract void com.usbcommander.config.contract.IMachineConfig.enableServerService () [abstract]
```

Este método se utiliza para habilitar la conexión con el servidor

Reimplemented in **com.usbcommander.config.MachineConfig** (p. 38).

6.9.2.2 getIp()

```
abstract String com.usbcommander.config.contract.IMachineConfig.getIp () [abstract]
```

Este método devuelve la ip del servidor configurado

Returns

Reimplemented in **com.usbcommander.config.MachineConfig** (p. 39).

6.9.2.3 getLogFrequency()

```
abstract long com.usbcommander.config.contract.IMachineConfig.getLogFrequency () [abstract]
```

Este método devuelve la frecuencia almacenada en memoria con la que se envían los registros automáticos

Returns

Reimplemented in **com.usbcommander.config.MachineConfig** (p. 39).

6.9.2.4 getPort()

```
abstract int com.usbcommander.config.contract.IMachineConfig.getPort () [abstract]
```

Este método devuelve el puerto del servidor configurado

Returns

Reimplemented in **com.usbcommander.config.MachineConfig** (p. 39).

6.9.2.5 isUsbEnable()

```
abstract boolean com.usbcommander.config.contract.IMachineConfig.isUsbEnable () [abstract]
```

Este método tiene como finalidad saber si debería ser posible montar unidades de memoria usb extraibles en la máquina

Returns

Reimplemented in **com.usbcommander.config.MachineConfig** (p. 39).

6.9.2.6 saveConfig()

```
abstract boolean com.usbcommander.config.contract.IMachineConfig.saveConfig () [abstract]
```

Este método guarda la configuración actualmente en memoria

Returns

Reimplemented in **com.usbcommander.config.MachineConfig** (p. 40).

6.9.2.7 setLogFrequency()

```
abstract void com.usbcommander.config.contract.IMachineConfig.setLogFrequency (
    long logFrequency) [abstract]
```

Este método se utiliza para configurar la frecuencia con la que la máquina debería generar los registros automáticos

Parameters

<i>logFrequency</i>	La nueva frecuencia en milesimas de segundo
---------------------	---

Reimplemented in **com.usbcommander.config.MachineConfig** (p. 40).

6.9.2.8 setUsbEnable()

```
abstract void com.usbcommander.config.contract.IMachineConfig.setUsbEnable (
    boolean usbEnable) [abstract]
```

Este método habilita o deshabilita la configuración de si una unidad usb de memoria extraíble debería de poder montarse o no

Parameters

<i>usbEnable</i>	Si se deberían poder (true) o no (false) poder montar memorias usb
------------------	--

Reimplemented in `com.usbcommander.config.MachineConfig` (p. 40).

6.9.3 Member Data Documentation

6.9.3.1 instance

```
IMachineConfig com.usbcommander.config.contract.IMachineConfig.instance [static], [protected]
```

Almacena la instancia de configuración usada en la aplicación. Este será inicializado por las clases que definan los métodos de la clase abstracta

6.10 com.usbcommander.managers.contract.IStatusManager Class Reference

Public Member Functions

- abstract **LogDTO generateLog ()**
- abstract **LogDTO generateLog (LogType type)**
- abstract **LogDTO generateLog (LogType type, String message)**
- abstract List< **LogDTO** > **getHistory ()**
- abstract void **deleteHistory ()**

Protected Member Functions

- **IStatusManager ()**

Static Protected Attributes

- static **IStatusManager instance**

6.10.1 Detailed Description

Clase abstracta usada como base para la creación de las clases encargadas de crear los registros del estado de la máquina

6.10.2 Constructor & Destructor Documentation

6.10.2.1 IStatusManager()

`com.usbcommander.managers.contract.IStatusManager.IStatusManager () [protected]`

6.10.3 Member Function Documentation

6.10.3.1 deleteHistory()

`abstract void com.usbcommander.managers.contract.IStatusManager.deleteHistory () [abstract]`

Método utilizado para eliminar los registros almacenados

Reimplemented in **com.usbcommander.managers.StatusManager** (p. 53).

6.10.3.2 generateLog() [1/3]

`abstract LogDTO com.usbcommander.managers.contract.IStatusManager.generateLog () [abstract]`

Método utilizado para generar y almacenar un registro automáticamente, evaluando el estado de la máquina y su estado esperado/deseado

Returns

El registro generado

Reimplemented in **com.usbcommander.managers.StatusManager** (p. 53).

6.10.3.3 generateLog() [2/3]

`abstract LogDTO com.usbcommander.managers.contract.IStatusManager.generateLog (LogType type) [abstract]`

Método utilizado para generar y almacenar un registro en función de un tipo definido

Parameters

<i>type</i>	El tipo del registro a crear
-------------	------------------------------

Returns

El registro generado

Reimplemented in **com.usbcommander.managers.StatusManager** (p. 54).

6.10.3.4 generateLog() [3/3]

```
abstract LogDTO com.usbcommander.managers.contract.IStatusManager.generateLog (
    LogType type,
    String message) [abstract]
```

Método utilizado para generar y almacenar un registro en función de un tipo definido y un mensaje específico. utilizado específicamente para los registros de error

Parameters

<i>type</i>	El tipo del registro a crear
<i>message</i>	El mensaje a almacenar en el registro

Returns

El registro generado

Reimplemented in **com.usbcommander.managers.StatusManager** (p. 54).

6.10.3.5 getHistory()

```
abstract List< LogDTO > com.usbcommander.managers.contract.IStatusManager.getHistory ()
[abstract]
```

Método utilizado para obtener todos los registros almacenados

Returns

Una lista con todos los registros actualmente almacenados

Reimplemented in **com.usbcommander.managers.StatusManager** (p. 54).

6.10.4 Member Data Documentation

6.10.4.1 instance

```
IStatusManager com.usbcommander.managers.contract.IStatusManager.instance [static], [protected]
```

Almacena la instancia de la clase usada en la aplicación. Este será inicializado por las clases que definan los métodos de la clase abstracta

6.11 com.usbcommander.managers.contract.IUsbMemoryManager Class Reference

Public Member Functions

- abstract List< Map< String, String > > **getConnectedDrives** ()
- abstract int **isDriveConnected** ()
- abstract int **getAccessValue** ()
- abstract void **enableAccess** ()
- abstract void **disableAccess** ()
- abstract void **removeExternalDrives** ()

Protected Member Functions

- **IUsbMemoryManager** ()

Static Protected Attributes

- static **IUsbMemoryManager** instance

6.11.1 Detailed Description

Clase abstracta usada como base para la creación de las clases encargadas de manipular la máquina para interactuar con las memorias usb

6.11.2 Constructor & Destructor Documentation

6.11.2.1 IUsbMemoryManager()

```
com.usbcommander.managers.contract.IUsbMemoryManager.IUsbMemoryManager () [protected]
```

6.11.3 Member Function Documentation

6.11.3.1 disableAccess()

```
abstract void com.usbcommander.managers.contract.IUsbMemoryManager.disableAccess () [abstract]
```

Este método sirve para prohibir la conexión de memorias usb en la máquina

Reimplemented in **com.usbcommander.managers.UsbMemoryManager** (p. 59).

6.11.3.2 enableAccess()

```
abstract void com.usbcommander.managers.contract.IUsbMemoryManager.enableAccess () [abstract]
```

Este método sirve para permitir la conexión de memorias usb en la máquina

Reimplemented in **com.usbcommander.managers.UsbMemoryManager** (p. 59).

6.11.3.3 getAccessValue()

```
abstract int com.usbcommander.managers.contract.IUsbMemoryManager.getAccessValue () [abstract]
```

Este método revisa si la máquina permite o no la conexión de memorias usb

Returns

Un int que define si es posible o no conectar memorias usb

Reimplemented in **com.usbcommander.managers.UsbMemoryManager** (p. 59).

6.11.3.4 getConnectedDrives()

```
abstract List< Map< String, String > > com.usbcommander.managers.contract.IUsbMemoryManager.getConnectedDrives () [abstract]
```

Este método se encarga de revisar las unidades de almacenamiento usb conectadas a la máquina y devuelve una lista con el nombre, la letra asignada a la unidad y el serial de cada unidad

Returns

Una lista con el nombre, la letra asignada a la unidad y el serial de cada unidad conectada a la máquina

Reimplemented in **com.usbcommander.managers.UsbMemoryManager** (p. 60).

6.11.3.5 isDriveConnected()

```
abstract int com.usbcommander.managers.contract.IUsbMemoryManager.isDriveConnected () [abstract]
```

Revisa si existe alguna unidad de memoria usb conectada a la máquina

Returns

El número de memorias conectadas a la máquina

Reimplemented in **com.usbcommander.managers.UsbMemoryManager** (p. 60).

6.11.3.6 removeExternalDrives()

```
abstract void com.usbcommander.managers.contract.IUsbMemoryManager.removeExternalDrives () [abstract]
```

Este método tiene como finalidad, desmontar todas las unidades de memoria usb conectadas a la máquina

Reimplemented in **com.usbcommander.managers.UsbMemoryManager** (p. 61).

6.11.4 Member Data Documentation

6.11.4.1 instance

```
IUsbMemoryManager com.usbcommander.managers.contract.IUsbMemoryManager.instance [static], [protected]
```

Almacena la instancia de la clase usada en la aplicación. Este será inicializado por las clases que definan los métodos de la clase abstracta

6.12 com.usbcommander.dto.LogDTO Class Reference

Public Member Functions

- **LogDTO ()**
- **LogDTO** (int **usbValue**, boolean **usbAllowed**, List< Map< String, String > > **usbList**, LocalDateTime **creationDate**, int **code**)
- **LogDTO** (int **code**, String **errorMessage**, LocalDateTime **creationDate**)
- int **getUsbValue** ()
- boolean **isUsbAllowed** ()
- List< Map< String, String > > **getUsbList** ()
- LocalDateTime **getCreationDate** ()
- int **getCode** ()
- void **setUsbValue** (int **usbValue**)
- void **setUsbAllowed** (boolean **usbAllowed**)
- void **setUsbList** (List< Map< String, String > > **usbList**)
- void **setCreationDate** (LocalDateTime **creationDate**)
- void **setCode** (int **code**)
- String **getErrorMessage** ()
- void **setErrorMessage** (String **errorMessage**)

Private Attributes

- int **usbValue**
- boolean **usbAllowed**
- List< Map< String, String > > **usbList**
- LocalDateTime **creationDate**
- String **errorMessage**
- int **code**

6.12.1 Detailed Description

Clase usada únicamente como DTO (Data transfer object) para ser enviada y recibida en comunicaciones entre el cliente y el servidor, empleada para que el cliente pueda enviar la información de los registros al servidor

6.12.2 Constructor & Destructor Documentation

6.12.2.1 LogDTO() [1/3]

```
com.usbcommander.dto.LogDTO.LogDTO ()
```

6.12.2.2 LogDTO() [2/3]

```
com.usbcommander.dto.LogDTO.LogDTO (
    int usbValue,
    boolean usbAllowed,
    List< Map< String, String > > usbList,
    LocalDateTime creationDate,
    int code)
```


6.12.2.3 LogDTO() [3/3]

```
com.usbcommander.dto.LogDTO.LogDTO (
    int code,
    String errorMessage,
    LocalDateTime creationDate)
```

6.12.3 Member Function Documentation

6.12.3.1 getCode()

```
int com.usbcommander.dto.LogDTO.getCode ()
```

6.12.3.2 getCreationDate()

```
LocalDateTime com.usbcommander.dto.LogDTO.getCreationDate ()
```

6.12.3.3 getErrorMessage()

```
String com.usbcommander.dto.LogDTO.getErrorMessage ()
```

6.12.3.4 getUsbList()

```
List< Map< String, String > > com.usbcommander.dto.LogDTO.getUsbList ()
```

6.12.3.5 getUsbValue()

```
int com.usbcommander.dto.LogDTO.getUsbValue ()
```

6.12.3.6 isUsbAllowed()

```
boolean com.usbcommander.dto.LogDTO.isUsbAllowed ()
```

6.12.3.7 setCode()

```
void com.usbcommander.dto.LogDTO.setCode (
    int code)
```

6.12.3.8 setCreationDate()

```
void com.usbcommander.dto.LogDTO.setCreationDate (
    LocalDateTime creationDate)
```

6.12.3.9 `setErrorMessage()`

```
void com.usbcommander.dto.LogDTO.setErrorMessage (  
    String errorMessage)
```

6.12.3.10 `setUsbAllowed()`

```
void com.usbcommander.dto.LogDTO.setUsbAllowed (  
    boolean usbAllowed)
```

6.12.3.11 `setUsbList()`

```
void com.usbcommander.dto.LogDTO.setUsbList (  
    List< Map< String, String > > usbList)
```

6.12.3.12 `setUsbValue()`

```
void com.usbcommander.dto.LogDTO.setUsbValue (  
    int usbValue)
```

6.12.4 Member Data Documentation

6.12.4.1 `code`

```
int com.usbcommander.dto.LogDTO.code [private]
```

6.12.4.2 `creationDate`

```
LocalDateTime com.usbcommander.dto.LogDTO.creationDate [private]
```

6.12.4.3 `errorMessage`

```
String com.usbcommander.dto.LogDTO.errorMessage [private]
```

6.12.4.4 `usbAllowed`

```
boolean com.usbcommander.dto.LogDTO.usbAllowed [private]
```

6.12.4.5 `usbList`

```
List<Map<String,String> > com.usbcommander.dto.LogDTO.usbList [private]
```

6.12.4.6 usbValue

```
int com.usbcommander.dto.LogDTO.usbValue [private]
```

6.13 com.usbcommander.services.LogService Class Reference

Public Member Functions

- void **run** ()
- void **sendLogs** () throws **ServiceDisabledException**

Public Member Functions inherited from com.usbcommander.services.contract.CommanderService

- synchronized void **start** ()
- void **stopService** ()

Static Public Member Functions

- static **LogService** **getInstance** ()

Private Member Functions

- **LogService** ()

Private Attributes

- **IStatusManager** **statusManager**
- **IMachineConfig** **machineConfig**
- **ObjectMapper** **mapper**

Static Private Attributes

- static **LogService** **instance**

Additional Inherited Members

Protected Member Functions inherited from com.usbcommander.services.contract.CommanderService

- **CommanderService** ()

Protected Attributes inherited from com.usbcommander.services.contract.CommanderService

- boolean **running**

6.13.1 Detailed Description

Servicio encargado de controlar la creación y envío de los registros de la máquina

6.13.2 Constructor & Destructor Documentation

6.13.2.1 LogService()

```
com.usbcommander.services.LogService.LogService () [private]
```

Constructor encargado de inicializar las propiedades del objeto

6.13.3 Member Function Documentation

6.13.3.1 getInstance()

```
LogService com.usbcommander.services.LogService.getInstance () [static]
```

Método estático usado para inicializar y obtener la instancia de este servicio

Returns

La instancia inicializada del Servicio

6.13.3.2 run()

```
void com.usbcommander.services.LogService.run ()
```

Reimplemented from **com.usbcommander.services.contract.CommanderService** (p. 17).

6.13.3.3 sendLogs()

```
void com.usbcommander.services.LogService.sendLogs () throws ServiceDisabledException
```

Método empleado para llevar a cabo el envío de todos los registros actualmente almacenados en el cliente

Exceptions

ServiceDisabledException (p. 52)	En el caso de que al momento de llamarse el método, este servicio no este activo
---	--

6.13.4 Member Data Documentation

6.13.4.1 instance

```
LogService com.usbcommander.services.LogService.instance [static], [private]
```

Instancia del servicio

6.13.4.2 machineConfig

IMachineConfig com.usbcommander.services.LogService.machineConfig [private]

Instancia de **IMachineConfig** (p. 22) para la obtención de las variables de configuración de la máquina

6.13.4.3 mapper

ObjectMapper com.usbcommander.services.LogService.mapper [private]

Instancia de jackson para la serialización de registros como json

6.13.4.4 statusManager

IStatusManager com.usbcommander.services.LogService.statusManager [private]

Instancia de **IStatusManager** (p. 25) para la creación y almacenaje de registros

6.14 com.usbcommander.enums.LogType Enum Reference

Public Member Functions

- int **getCode** ()

Public Attributes

- **INFO** =(1001)
- **CONFIG_MOD** =(1002)
- **REGISTRY_MOD** =(1003)
- **INCOHERENT** =(1004)
- **MEMORY_CONN** =(1005)
- **CONNECTION** =(1006)
- **ERROR** =(1007)

Private Member Functions

- **LogType** (int code)

Private Attributes

- int **code**

6.14.1 Detailed Description

Enum en el que se definen los diferentes tipos de registros con sus respectivos códigos identificativos

6.14.2 Constructor & Destructor Documentation

6.14.2.1 LogType()

```
com.usbcommander.enums.LogType.LogType (  
    int code) [private]
```

6.14.3 Member Function Documentation

6.14.3.1 getCode()

```
int com.usbcommander.enums.LogType.getCode ()
```

6.14.4 Member Data Documentation

6.14.4.1 code

```
int com.usbcommander.enums.LogType.code [private]
```

6.14.4.2 CONFIG_MOD

```
com.usbcommander.enums.LogType.CONFIG_MOD =(1002)
```

6.14.4.3 CONNECTION

```
com.usbcommander.enums.LogType.CONNECTION =(1006)
```

6.14.4.4 ERROR

```
com.usbcommander.enums.LogType.ERROR =(1007)
```

6.14.4.5 INCOHERENT

```
com.usbcommander.enums.LogType.INCOHERENT =(1004)
```

6.14.4.6 INFO

```
com.usbcommander.enums.LogType.INFO =(1001)
```

6.14.4.7 MEMORY_CONN

```
com.usbcommander.enums.LogType.MEMORY_CONN =(1005)
```

6.14.4.8 REGISTRY_MOD

```
com.usbcommander.enums.LogType.REGISTRY_MOD = (1003)
```

6.15 com.usbcommander.config.MachineConfig Class Reference

Public Member Functions

- boolean **saveConfig** ()
- void **enableServerService** ()
- boolean **isUsbEnable** ()
- long **getLogFrequency** ()
- String **getIp** ()
- int **getPort** ()
- void **setUsbEnable** (boolean **usbEnable**)
- void **setLogFrequency** (long **logFrequency**)

Static Public Member Functions

- static **IMachineConfig getInstance** ()

Private Member Functions

- **MachineConfig** ()
- **MachineConfig** (boolean **usbEnable**, long **logFrequency**)

Private Attributes

- boolean **usbEnable**
- long **logFrequency**
- String **ip**
- Integer **port**

Additional Inherited Members

Static Protected Attributes inherited from com.usbcommander.config.contract.IMachineConfig

- static **IMachineConfig instance**

6.15.1 Detailed Description

Implementación de **IMachineConfig** (p. 22) para máquinas con windows como sistema operativo

6.15.2 Constructor & Destructor Documentation

6.15.2.1 MachineConfig() [1/2]

```
com.usbcommander.config.MachineConfig.MachineConfig () [private]
```

Constructor usado cuando no se encuentra configuración previa. Almacena los valores por defecto de configuración de la aplicación en el registro de windows

6.15.2.2 MachineConfig() [2/2]

```
com.usbcommander.config.MachineConfig.MachineConfig (  
    boolean usbEnable,  
    long logFrequency) [private]
```

Constructor usado cuándo se encuentran valores almacenados de configuración de la aplicación en el registro de windows

Parameters

usbEnable (p. 41)	Si se deberían poder (true) o no (false) poder montar memorias usb
logFrequency (p. 41)	La frecuencia en milisegundos con la que se envían los registros automáticos

6.15.3 Member Function Documentation

6.15.3.1 enableServerService()

```
void com.usbcommander.config.MachineConfig.enableServerService ()
```

Este método se utiliza para habilitar la conexión con el servidor

Reimplemented from **com.usbcommander.config.contract.IMachineConfig** (p. 23).

6.15.3.2 getInstance()

```
IMachineConfig com.usbcommander.config.MachineConfig.getInstance () [static]
```

Método estático que construye una instancia de **IMachineConfig** (p. 22) con la implementación definida en esta clase. En función de si encuentra o no los valores necesarios en el registro de windows, hará uso de un constructor u otro.

Returns

La instancia almacenada en la clase abstracta, inicializada como objeto de esta clase

6.15.3.3 getIp()

```
String com.usbcommander.config.MachineConfig.getIp ()
```

Este método devuelve la ip del servidor configurado

Returns

Reimplemented from **com.usbcommander.config.contract.IMachineConfig** (p. 23).

6.15.3.4 getLogFrecuency()

```
long com.usbcommander.config.MachineConfig.getLogFrecuency ()
```

Este método devuelve la frecuencia almacenada en memoria con la que se envían los registros automáticos

Returns

Reimplemented from **com.usbcommander.config.contract.IMachineConfig** (p. 23).

6.15.3.5 getPort()

```
int com.usbcommander.config.MachineConfig.getPort ()
```

Este método devuelve el puerto del servidor configurado

Returns

Reimplemented from **com.usbcommander.config.contract.IMachineConfig** (p. 23).

6.15.3.6 isUsbEnable()

```
boolean com.usbcommander.config.MachineConfig.isUsbEnable ()
```

Este método tiene como finalidad saber si debería ser posible montar unidades de memoria usb extraíbles en la máquina

Returns

Reimplemented from **com.usbcommander.config.contract.IMachineConfig** (p. 24).

6.15.3.7 saveConfig()

```
boolean com.usbcommander.config.MachineConfig.saveConfig ()
```

Este método guarda la configuración actualmente en memoria

Returns

Reimplemented from **com.usbcommander.config.contract.IMachineConfig** (p. 24).

6.15.3.8 setLogFrequency()

```
void com.usbcommander.config.MachineConfig.setLogFrequency (  
    long logFrequency)
```

Este método se utiliza para configurar la frecuencia con la que la máquina debería generar los registros automáticos

Parameters

<i>logFrequency</i> (p. 41)	La nueva frecuencia en milesimas de segundo
------------------------------------	---

Reimplemented from **com.usbcommander.config.contract.IMachineConfig** (p. 24).

6.15.3.9 setUsbEnable()

```
void com.usbcommander.config.MachineConfig.setUsbEnable (  
    boolean usbEnable)
```

Este método habilita o deshabilita la configuración de si una unidad usb de memoria extraible debería de poder montarse o no

Parameters

<i>usbEnable</i> (p. 41)	Si se deberían poder (true) o no (false) poder montar memorias usb
---------------------------------	--

Reimplemented from **com.usbcommander.config.contract.IMachineConfig** (p. 25).

6.15.4 Member Data Documentation

6.15.4.1 ip

```
String com.usbcommander.config.MachineConfig.ip [private]
```

Valor que almacena la ip del servidor

6.15.4.2 logFrequency

```
long com.usbcommander.config.MachineConfig.logFrequency [private]
```

Valor que almacena la frecuencia con la que se envían los registros automáticos

6.15.4.3 port

```
Integer com.usbcommander.config.MachineConfig.port [private]
```

Valor que almacena el puerto del servidor

6.15.4.4 usbEnable

```
boolean com.usbcommander.config.MachineConfig.usbEnable [private]
```

Valor que almacena si debería ser posible montar unidades de memoria usb extraíbles en la máquina

6.16 com.usbcommander.errors.NotConnectionException Class Reference

Public Member Functions

- **NotConnectionException** (String mess)

6.16.1 Detailed Description

Excepción definida para ser lanzada en caso de que un método de la clase ServerConnectionService sea llamado mientras que no haya una conexión establecida con el servidor

6.16.2 Constructor & Destructor Documentation

6.16.2.1 NotConnectionException()

```
com.usbcommander.errors.NotConnectionException.NotConnectionException (  
    String mess)
```

6.17 com.usbcommander.AppConst.RegistryReferences Class Reference

Static Public Attributes

- static final int **ENABLE_VALUE** = 3
- static final int **DISABLE_VALUE** = 4
- static final String **USB_ENUM** = "SYSTEM\\CurrentControlSet\\Services\\USBSTOR\\Enum"
- static final String **USB_STOR** = "SYSTEM\\CurrentControlSet\\Services\\USBSTOR"

6.17.1 Member Data Documentation

6.17.1.1 DISABLE_VALUE

```
final int com.usbcommander.AppConst.RegistryReferences.DISABLE_VALUE = 4 [static]
```

6.17.1.2 ENABLE_VALUE

```
final int com.usbcommander.AppConst.RegistryReferences.ENABLE_VALUE = 3 [static]
```

6.17.1.3 USB_ENUM

```
final String com.usbcommander.AppConst.RegistryReferences.USB_ENUM = "SYSTEM\\CurrentControl←  
Set\\Services\\USBSTOR\\Enum" [static]
```

6.17.1.4 USB_STOR

```
final String com.usbcommander.AppConst.RegistryReferences.USB_STOR = "SYSTEM\\CurrentControl←  
Set\\Services\\USBSTOR" [static]
```

6.18 com.usbcommander.services.SecurityService Class Reference

Public Member Functions

- void **openAccess** (long time) throws **ServiceDisabledException**
- void **forceClose** () throws **ServiceDisabledException**
- void **changeLogFrequency** (long frequency)
- void **run** ()

Public Member Functions inherited from com.usbcommander.services.contract.CommanderService

- synchronized void **start** ()
- void **stopService** ()

Static Public Member Functions

- static **SecurityService** **getInstance** ()

Private Member Functions

- **SecurityService** ()
- boolean **usbStorMatchesMemory** ()
- boolean **configMatchesMemory** ()
- Thread **setListenerOn** (String route, Runnable onChange, Consumer< Win32Exception > onCatch)
- boolean **startListener** (Thread listener)

Private Attributes

- Thread **usbStorListener**
- Thread **appConfigListener**
- **IUsbMemoryManager** **usbMemoryManager**
- **IStatusManager** **statusManager**
- **IMachineConfig** **machineConfig**
- long **grantedFor**
- Thread **grantedAccess**

Static Private Attributes

- static **SecurityService** **instance**

Additional Inherited Members

Protected Member Functions inherited from **com.usbcommander.services.contract.CommanderService**

- **CommanderService** ()

Protected Attributes inherited from **com.usbcommander.services.contract.CommanderService**

- boolean **running**

6.18.1 Detailed Description

Servicio encargado de controlar y asegurar las funciones de seguridad que controlan el uso de memorias usb y las correctas modificaciones en los archivos de configuración de la aplicación Por el momento, su implementación es específica de Windows

6.18.2 Constructor & Destructor Documentation

6.18.2.1 SecurityService()

```
com.usbcommander.services.SecurityService.SecurityService () [private]
```

Constructor encargado de inicializar las propiedades del objeto

6.18.3 Member Function Documentation

6.18.3.1 changeLogFrequency()

```
void com.usbcommander.services.SecurityService.changeLogFrequency (  
    long frequency)
```

6.18.3.2 configMatchesMemory()

```
boolean com.usbcommander.services.SecurityService.configMatchesMemory () [private]
```

Compara los valores de configuración almacenados en memoria con los almacenados en la máquina

Returns

True si los valores coinciden, false en el caso opuesto

6.18.3.3 forceClose()

```
void com.usbcommander.services.SecurityService.forceClose () throws ServiceDisabledException
```

Este método deshabilita la posibilidad de conectar memorias usb y eyecta las actualmente montadas

Returns

Un booleano representando si la operación tuvo exito o no

Exceptions

<i>ServiceDisabledException</i> (p. 52)	En el caso de que al momento de llamarse el método, este servicio no este activo
--	--

6.18.3.4 getInstance()

```
SecurityService com.usbcommander.services.SecurityService.getInstance () [static]
```

Método estático usado para inicializar y obtener la instancia de este servicio

Returns

La instancia inicializada del Servicio

6.18.3.5 openAccess()

```
void com.usbcommander.services.SecurityService.openAccess (  
    long time) throws ServiceDisabledException
```

Este método habilita la conexión de memorias usb en la máquina durante el tiempo establecido

Parameters

<i>time</i>	El tiempo durante el cuál será posible conectar memoras usb en la máquina
-------------	---

Exceptions

<i>ServiceDisabledException</i> (p. 52)	En el caso de que al momento de llamarse el método, este servicio no este activo
--	--

6.18.3.6 run()

```
void com.usbcommander.services.SecurityService.run ()
```

Reimplemented from **com.usbcommander.services.contract.CommanderService** (p. 17).

6.18.3.7 setListenerOn()

```
Thread com.usbcommander.services.SecurityService.setListenerOn (  
    String route,  
    Runnable onChange,  
    Consumer< Win32Exception > onCatch) [private]
```

Método empleador para facilitar la creación de Threads encargados de supervisar entradas en el registro de windows

Parameters

<i>route</i>	La ruta del valor del registro en la cuál se va a crear el hilo de supervisión
<i>onChange</i>	La función a ejecutar cuando se detectan cambios en el registro
<i>onCatch</i>	La función a ejecutar si se lanza una excepción al momento de lanzar el hilo

Returns

El hilo de supervisión

6.18.3.8 startListener()

```
boolean com.usbcommander.services.SecurityService.startListener (  
    Thread listener) [private]
```

Método usado para iniciar la ejecución de los Thread creados en el Servicio para la supervisión de los campos en el registro de windows

Parameters

<i>listener</i>	El Thread de supervisión a iniciar
-----------------	------------------------------------

Returns

Un booleano indicando si la operación tuvo éxito o no

6.18.3.9 usbStorMatchesMemory()

```
boolean com.usbcommander.services.SecurityService.usbStorMatchesMemory () [private]
```

Compara el estado actual del permiso de uso de las memorias usb en el registro de windows con el valor de configuración de la aplicación almacenado en memoria

Returns

True si el registro coincide con el valor de configuración, y falso en el caso contrario

6.18.4 Member Data Documentation

6.18.4.1 appConfigListener

```
Thread com.usbcommander.services.SecurityService.appConfigListener [private]
```

Objeto Thread que almacena las instrucciones a ejecutar en el caso de que se produzca una modificación en los archivos de configuración de la aplicación

6.18.4.2 grantedAccess

```
Thread com.usbcommander.services.SecurityService.grantedAccess [private]
```

Objeto Thread que almacena las instrucciones a ejecutar al momento de posibilitar el uso de memorias usb

6.18.4.3 grantedFor

```
long com.usbcommander.services.SecurityService.grantedFor [private]
```

Variable que representa el tiempo de acceso permitido para el uso de memorias usb

6.18.4.4 instance

```
SecurityService com.usbcommander.services.SecurityService.instance [static], [private]
```

Instancia del servicio

6.18.4.5 machineConfig

```
IMachineConfig com.usbcommander.services.SecurityService.machineConfig [private]
```

Instancia de **IMachineConfig** (p. 22) para obtener la configuración esperada, así como modificarla

6.18.4.6 statusManager

IStatusManager com.usbcommander.services.SecurityService.statusManager [private]

Instancia de **IStatusManager** (p. 25) para la creación de registros en el caso de que se detecten modificaciones no permitidas en la máquina

6.18.4.7 usbMemoryManager

IUsbMemoryManager com.usbcommander.services.SecurityService.usbMemoryManager [private]

Instancia de **IUsbMemoryManager** (p. 27) para controlar el estado de las memorias usb con respecto a la máquina

6.18.4.8 usbStorListener

Thread com.usbcommander.services.SecurityService.usbStorListener [private]

Objeto Thread que almacena las instrucciones a ejecutar en el caso de que se produzca una modificación en el registro de windows

6.19 com.usbcommander.services.ServerConnectionService Class Reference

Public Member Functions

- void **run** ()
- boolean **sendMessage** (String message) throws **ServiceDisabledException**, **NotConnectionException**

Public Member Functions inherited from com.usbcommander.services.contract.CommanderService

- synchronized void **start** ()
- void **stopService** ()

Static Public Member Functions

- static **ServerConnectionService getInstance** ()

Private Member Functions

- **ServerConnectionService** ()
- void **connectToServer** ()
- Thread **listenerThread** ()

Private Attributes

- final String **SENDING_ERROR** = "Error while sending server message: "
- final String **SERVER_MSG_ERROR** = "Error while reading server message"
- final String **CONN_ERROR** = "Error while trying to stablsh connection. Error Info: "
- final String **MAX_RETRY_MSG** = "Max connection attemps reached. Connection will be disable this session"
- final int **MAX_RETRY** = 5
- final long **RECONNECT_TIMEOUT** = 30000
- Socket **socket**
- **IMachineConfig** **machineConfig**
- **IStatusManager** **statusManager**
- boolean **connected**
- ObjectMapper **mapper**

Static Private Attributes

- static **ServerConnectionService** **instance**

Additional Inherited Members

Protected Member Functions inherited from `com.usbcommander.services.contract.CommanderService`

- `CommanderService ()`

Protected Attributes inherited from `com.usbcommander.services.contract.CommanderService`

- boolean **running**

6.19.1 Detailed Description

Servicio encargado de contolar la conexión con el servidor

6.19.2 Constructor & Destructor Documentation

6.19.2.1 `ServerConnectionService()`

```
com.usbcommander.services.ServerConnectionService.ServerConnectionService () [private]
```

Constructor encargado de inicializar las propiedades del objeto

6.19.3 Member Function Documentation

6.19.3.1 `connectToServer()`

```
void com.usbcommander.services.ServerConnectionService.connectToServer () [private]
```

Método usado para llevar a cabo un intento de conexión con el servidor

6.19.3.2 getInstance()

```
ServerConnectionService com.usbcommander.services.ServerConnectionService.getInstance ()  
[static]
```

Método estático usado para inicializar y obtener la instancia de este servicio

Returns

La instancia inicializada del Servicio

6.19.3.3 listenerThread()

```
Thread com.usbcommander.services.ServerConnectionService.listenerThread () [private]
```

Método encargado de generar un hilo que escuche constantemente un mensaje del servidor para interpretarlo, llevando a cabo las instrucciones necesarias para cumplir aquellas descritas en el mensaje.

Returns

6.19.3.4 run()

```
void com.usbcommander.services.ServerConnectionService.run ()
```

Reimplemented from **com.usbcommander.services.contract.CommanderService** (p. 17).

6.19.3.5 sendMessage()

```
boolean com.usbcommander.services.ServerConnectionService.sendMessage (  
    String message) throws ServiceDisabledException, NotConnectionException
```

Método cuya función es la de enviar el mensaje descrito por parámetro al servidor

Parameters

<i>message</i>	El mensaje a enviar al servidor
----------------	---------------------------------

Returns

Un booleano que expresa si la operación a tenido éxito o no

Exceptions

<i>ServiceDisabledException</i> (p. 52)	En el caso de que al momento de llamarse el método, este servicio no este activo
<i>NotConnectionException</i> (p. 41)	En el caso de que al momento de llamarse el método no exista una conexión con el servidor

6.19.4 Member Data Documentation**6.19.4.1 CONN_ERROR**

```
final String com.usbcommander.services.ServerConnectionService.CONN_ERROR = "Error while trying
to stablish connection. Error Info: " [private]
```

Mensaje de error a usar cuándo se produce un error al establecer la conexión con el servidor

6.19.4.2 connected

```
boolean com.usbcommander.services.ServerConnectionService.connected [private]
```

Variable encargada de almacenar si existe o no una conexión con el servidor

6.19.4.3 instance

```
ServerConnectionService com.usbcommander.services.ServerConnectionService.instance [static],
[private]
```

Instancia del servicio

6.19.4.4 machineConfig

```
IMachineConfig com.usbcommander.services.ServerConnectionService.machineConfig [private]
```

Instancia de **IMachineConfig** (p. 22) para obtener la dirección del servidor

6.19.4.5 mapper

```
ObjectMapper com.usbcommander.services.ServerConnectionService.mapper [private]
```

Instancia de jackson para serializar la información a enviar al servidor y deserializar la información enviada por este

6.19.4.6 MAX_RETRY

```
final int com.usbcommander.services.ServerConnectionService.MAX_RETRY = 5 [private]
```

Número máximo de reintentos de conexión

6.19.4.7 MAX_RETRY_MSG

```
final String com.usbcommander.services.ServerConnectionService.MAX_RETRY_MSG = "Max connection  
attempts reached. Connection will be disable this session" [private]
```

Mensaje de error a usar cuándo se llega al número máximo de reintentos de conexión

6.19.4.8 RECONNECT_TIMEOUT

```
final long com.usbcommander.services.ServerConnectionService.RECONNECT_TIMEOUT = 30000 [private]
```

Tiempo a esperar entre reintentos de conexión

6.19.4.9 SENDING_ERROR

```
final String com.usbcommander.services.ServerConnectionService.SENDING_ERROR = "Error while  
sending server message: " [private]
```

Mensaje de error a usar cuándo se produce un error al enviar información al servidor

6.19.4.10 SERVER_MSG_ERROR

```
final String com.usbcommander.services.ServerConnectionService.SERVER_MSG_ERROR = "Error while  
reading server message" [private]
```

Mensaje de error a usar cuándo se produce un error al leer un mensaje del servidor

6.19.4.11 socket

```
Socket com.usbcommander.services.ServerConnectionService.socket [private]
```

Objeto socket encargado de conectarse al servidor

6.19.4.12 statusManager

```
IStatusManager com.usbcommander.services.ServerConnectionService.statusManager [private]
```

Instancia de **IStatusManager** (p. 25) para la creación de registros de error

6.20 com.usbcommander.errors.ServiceDisabledException Class Reference

Public Member Functions

- **ServiceDisabledException** (String mess)

6.20.1 Detailed Description

Excepción definida para ser lanzada en caso de que se llame a un método de una de las clases que extiendan a CommanderService mientras que su respectivo hilo no este funcionando.

6.20.2 Constructor & Destructor Documentation

6.20.2.1 ServiceDisabledException()

```
com.usbcommander.errors.ServiceDisabledException.ServiceDisabledException (  
    String mess)
```

6.21 com.usbcommander.managers.StatusManager Class Reference

Public Member Functions

- **LogDTO generateLog** ()
- **LogDTO generateLog** (**LogType** type)
- **LogDTO generateLog** (**LogType** type, String message)
- List< **LogDTO** > **getHistory** ()
- void **deleteHistory** ()

Static Public Member Functions

- static **IStatusManager getInstance** ()

Private Member Functions

- **StatusManager** ()
- void **writeEventLog** (int type, int code, String message)

Private Attributes

- **IUsbMemoryManager usbMemoryManager**
- **IMachineConfig machineConfig**
- **ObjectMapper mapper**
- **Advapi32 advapi32**

Additional Inherited Members

Protected Member Functions inherited from com.usbcommander.managers.contract.IStatusManager

- `IStatusManager ()`

Static Protected Attributes inherited from com.usbcommander.managers.contract.IStatusManager

- static `IStatusManager` instance

6.21.1 Detailed Description

Implementación de IStatusManager para funciionarr en un entorno de windows haciendo uso del registro de windows y windows event viewer

6.21.2 Constructor & Destructor Documentation

6.21.2.1 StatusManager()

```
com.usbcommander.managers.StatusManager.StatusManager () [private]
```

Constructor de la clase en el que se inicializan las propiedades del objeto

6.21.3 Member Function Documentation

6.21.3.1 deleteHistory()

```
void com.usbcommander.managers.StatusManager.deleteHistory ()
```

Método utilizado para eliminar los registros almacenados

Reimplemented from `com.usbcommander.managers.contract.IStatusManager` (p. 26).

6.21.3.2 generateLog() [1/3]

```
LogDTO com.usbcommander.managers.StatusManager.generateLog ()
```

Método utilizado para generar y almacenar un registro automáticamente, evaluando el estado de la máquina y su estado esperado/deseado

Returns

El registro generado

Reimplemented from `com.usbcommander.managers.contract.IStatusManager` (p. 26).

6.21.3.3 generateLog() [2/3]

```
LogDTO com.usbcommander.managers.StatusManager.generateLog (  
    LogType type)
```

Método utilizado para generar y almacenar un registro en función de un tipo definido

Parameters

<i>type</i>	El tipo del registro a crear
-------------	------------------------------

Returns

El registro generado

Reimplemented from **com.usbcommander.managers.contract.IStatusManager** (p. 26).

6.21.3.4 generateLog() [3/3]

```
LogDTO com.usbcommander.managers.StatusManager.generateLog (  
    LogType type,  
    String message)
```

Método utilizado para generar y almacenar un registro en función de un tipo definido y un mensaje específico. utilizado específicamente para los registros de error

Parameters

<i>type</i>	El tipo del registro a crear
<i>message</i>	El mensaje a almacenar en el registro

Returns

El registro generado

Reimplemented from **com.usbcommander.managers.contract.IStatusManager** (p. 27).

6.21.3.5 getHistory()

```
List< LogDTO > com.usbcommander.managers.StatusManager.getHistory ()
```

Método utilizado para obtener todos los registros almacenados

Returns

Una lista con todos los registros actualmente almacenados

Reimplemented from **com.usbcommander.managers.contract.IStatusManager** (p. 27).

6.21.3.6 getInstance()

IStatusManager com.usbcommander.managers.StatusManager.getInstance () [static]

Método estático que construye una instancia de IStatusManager con la implementación definida en esta clase.

Returns

La instancia almacenada en la clase abstracta, inicializada como objeto de esta clase

6.21.3.7 writeEventLog()

```
void com.usbcommander.managers.StatusManager.writeEventLog (  
    int type,  
    int code,  
    String message) [private]
```

Método usado para facilitar la escritura de los objetos de registro en windows event viewer.

Parameters

<i>type</i>	El tipo de registro de windows a almacenar
<i>code</i>	El código de registro asignado por la aplicación
<i>message</i>	El mensaje a almacenar en registro de windows event viewer

6.21.4 Member Data Documentation

6.21.4.1 advapi32

Advapi32 com.usbcommander.managers.StatusManager.advapi32 [private]

Instancia de Advapi32 para interactuar con el registro de windows y windows event viewer

6.21.4.2 machineConfig

IMachineConfig com.usbcommander.managers.StatusManager.machineConfig [private]

Instancia de **IMachineConfig** (p. 22) para comparar el estado esperado de la máquina

6.21.4.3 mapper

ObjectMapper com.usbcommander.managers.StatusManager.mapper [private]

Instancia de Jackson para serializar objetos de registro como json

6.21.4.4 usbMemoryManager

IUsbMemoryManager `com.usbcommander.managers.StatusManager.usbMemoryManager` [private]

Instancia de **IUsbMemoryManager** (p. 27) usada para obtener información del estado de la máquina

6.22 com.usbcommander.services.ThreadService Class Reference

Public Member Functions

- `void run ()`

Public Member Functions inherited from `com.usbcommander.services.contract.CommanderService`

- `synchronized void start ()`
- `void stopService ()`

Static Public Member Functions

- `static ThreadService getInstance ()`

Private Member Functions

- `ThreadService ()`

Private Attributes

- `IMachineConfig machineConfig`
- `IStatusManager statusManager`

Static Private Attributes

- `static ThreadService instance`

Additional Inherited Members

Protected Member Functions inherited from `com.usbcommander.services.contract.CommanderService`

- `CommanderService ()`

Protected Attributes inherited from `com.usbcommander.services.contract.CommanderService`

- `boolean running`

6.22.1 Detailed Description

Servicio encargado de lanzar y controlar el estado de otros servicios

6.22.2 Constructor & Destructor Documentation

6.22.2.1 ThreadService()

```
com.usbcommander.services.ThreadService.ThreadService () [private]
```

Constructor encargado de inicializar las propiedades del objeto

6.22.3 Member Function Documentation

6.22.3.1 getInstance()

```
ThreadService com.usbcommander.services.ThreadService.getInstance () [static]
```

Método estático usado para inicializar y obtener la instancia de este servicio

Returns

La instancia inicializada del Servicio

6.22.3.2 run()

```
void com.usbcommander.services.ThreadService.run ()
```

Reimplemented from **com.usbcommander.services.contract.CommanderService** (p. 17).

6.22.4 Member Data Documentation

6.22.4.1 instance

```
ThreadService com.usbcommander.services.ThreadService.instance [static], [private]
```

Instancia del servicio

6.22.4.2 machineConfig

```
IMachineConfig com.usbcommander.services.ThreadService.machineConfig [private]
```

Instancia de **IMachineConfig** (p. 22) usada para habilitar la conexión con el servidor

6.22.4.3 statusManager

IStatusManager `com.usbcommander.services.ThreadService.statusManager` [private]

Instacia de **IStatusManager** (p. 25) usada para generar una entrada de registro al iniciar la aplicación

6.23 com.usbcommander.managers.UsbMemoryManager Class Reference

Public Member Functions

- `List< Map< String, String > >` **getConnectedDrives** ()
- `int` **isDriveConnected** ()
- `int` **getAccessValue** ()
- `void` **enableAccess** ()
- `void` **disableAccess** ()
- `void` **removeExternalDrives** ()

Static Public Member Functions

- `static IUsbMemoryManager` **getInstance** ()

Private Member Functions

- `UsbMemoryManager` ()
- `void` **removeDrive** (String drive)

Private Attributes

- `final int` **IOCTL_STORAGE_EJECT_MEDIA** = 0x2D4808

Additional Inherited Members

Protected Member Functions inherited from `com.usbcommander.managers.contract.IUsbMemoryManager`

- `IUsbMemoryManager` ()

Static Protected Attributes inherited from `com.usbcommander.managers.contract.IUsbMemoryManager`

- `static IUsbMemoryManager` **instance**

6.23.1 Detailed Description

Implementación de IUsbMemoryManager para funcionar en un entorno de windows, mediante modificaciones en el registro de windows y el uso de herramientas propias de este sistema operativo

6.23.2 Constructor & Destructor Documentation

6.23.2.1 UsbMemoryManager()

```
com.usbcommander.managers.UsbMemoryManager.UsbMemoryManager () [private]
```

6.23.3 Member Function Documentation

6.23.3.1 disableAccess()

```
void com.usbcommander.managers.UsbMemoryManager.disableAccess ()
```

Este método sirve para prohibir la conexión de memorias usb en la máquina

Reimplemented from **com.usbcommander.managers.contract.IUsbMemoryManager** (p. 28).

6.23.3.2 enableAccess()

```
void com.usbcommander.managers.UsbMemoryManager.enableAccess ()
```

Este método sirve para permitir la conexión de memorias usb en la máquina

Reimplemented from **com.usbcommander.managers.contract.IUsbMemoryManager** (p. 28).

6.23.3.3 getAccessValue()

```
int com.usbcommander.managers.UsbMemoryManager.getAccessValue ()
```

Este método revisa si la máquina permite o no la conexión de memorias usb

Returns

Un int que define si es posible o no conectar memorias usb

Reimplemented from **com.usbcommander.managers.contract.IUsbMemoryManager** (p. 28).

6.23.3.4 `getConnectedDrives()`

```
List< Map< String, String > > com.usbcommander.managers.UsbMemoryManager.getConnectedDrives
()
```

Este método se encarga de revisar las unidades de almacenamiento usb conectadas a la máquina y devuelve una lista con el nombre, la letra asignada a la unidad y el serial de cada unidad

Returns

Una lista con el nombre, la letra asignada a la unidad y el serial de cada unidad conectada a la máquina

Reimplemented from **com.usbcommander.managers.contract.IUsbMemoryManager** (p. 29).

6.23.3.5 `getInstance()`

```
IUsbMemoryManager com.usbcommander.managers.UsbMemoryManager.getInstance () [static]
```

Método estático que construye una instancia de `IUsbMemoryManager` con la implementación definida en esta clase.

Returns

La instancia almacenada en la clase abstracta, inicializada como objeto de esta clase

6.23.3.6 `isDriveConnected()`

```
int com.usbcommander.managers.UsbMemoryManager.isDriveConnected ()
```

Revisa si existe alguna unidad de memoria usb conectada a la máquina

Returns

El número de memorias conectadas a la máquina

Reimplemented from **com.usbcommander.managers.contract.IUsbMemoryManager** (p. 29).

6.23.3.7 `removeDrive()`

```
void com.usbcommander.managers.UsbMemoryManager.removeDrive (
    String drive) [private]
```

Este método es usado a la hora de desmontar las unidades de memoria usb. Se encarga de desmontar una unidad de memoria usb en función de la letra asignada a esta.

Parameters

<i>drive</i>	La letra a la que la memoria usb esta asignada
--------------	--

6.23.3.8 removeExternalDrives()

```
void com.usbcommander.managers.UsbMemoryManager.removeExternalDrives ()
```

Este método tiene como finalidad, desmontar todas las unidades de memoria usb conectadas a la máquina

Reimplemented from **com.usbcommander.managers.contract.IUsbMemoryManager** (p. 29).

6.23.4 Member Data Documentation

6.23.4.1 IOCTL_STORAGE_EJECT_MEDIA

```
final int com.usbcommander.managers.UsbMemoryManager.IOCTL_STORAGE_EJECT_MEDIA = 0x2D4808  
[private]
```

Esta constante representa un valor de windows no presente en la librería de jna empleado a la hora de desmontar una unidad de memoria usb.

Index

- advapi32
 - com.usbcommander.managers.StatusManager, 55
- allow
 - com.usbcommander.dto.ConfigDTO, 19
- allowedTime
 - com.usbcommander.dto.ConfigDTO, 19
- appConfigListener
 - com.usbcommander.services.SecurityService, 46
- changeLogFrequency
 - com.usbcommander.services.SecurityService, 43
- code
 - com.usbcommander.dto.LogDTO, 32
 - com.usbcommander.enums.LogType, 36
- com.usbcommander, 11
- com.usbcommander.App, 15
 - main, 15
- com.usbcommander.AppConst, 15
 - MAIN_LOCATION, 15
- com.usbcommander.AppConst.ConfigReferences, 19
 - CONFIG_LOCATION, 19
 - IP_ENTRY, 19
 - LOG_ENTRY, 19
 - MIN_FRECUENNCY, 19
 - PORT_ENTRY, 19
 - USB_ALLOW_ENTRY, 20
- com.usbcommander.AppConst.ErrorMessages, 20
 - CONNECTION_ERROR_MESSAGE, 20
 - ERROR_DELETE, 20
 - JACKSON_ERROR_TEXT, 20
 - LOG_GENERATION_ERROR, 20
 - SERVICE_NOT_RUNNING, 20
 - WEIRDVALUE_ERROR, 21
- com.usbcommander.AppConst.EventLogReferences,
21
 - ENTRY_NAME, 21
 - ENTRY_ROUTE, 21
 - ERROR_CODE, 21
 - ERROR_CONNECTION_CODE, 21
 - INCOHERENT_VALUE, 21
 - INFO_CODE, 22
 - MEMORY_CONNECTED, 22
 - REGISTRY_MODIFICATION, 22
 - UNAUTHORIZED_CONFIGURATION_MODIFICATION,
22
- com.usbcommander.AppConst.RegistryReferences, 41
 - DISABLE_VALUE, 42
 - ENABLE_VALUE, 42
 - USB_ENUM, 42
 - USB_STOR, 42
- com.usbcommander.AppTest, 16
 - shouldAnswerWithTrue, 16
- com.usbcommander.config, 11
- com.usbcommander.config.contract, 11
- com.usbcommander.config.contract.IMachineConfig, 22
 - enableServerService, 23
 - getIp, 23
 - getLogFrequency, 23
 - getPort, 23
 - instance, 25
 - isUsbEnable, 23
 - saveConfig, 24
 - setLogFrequency, 24
 - setUsbEnable, 24
- com.usbcommander.config.MachineConfig, 37
 - enableServerService, 38
 - getInstance, 38
 - getIp, 38
 - getLogFrequency, 39
 - getPort, 39
 - ip, 40
 - isUsbEnable, 39
 - logFrequency, 40
 - MachineConfig, 38
 - port, 41
 - saveConfig, 39
 - setLogFrequency, 40
 - setUsbEnable, 40
 - usbEnable, 41
- com.usbcommander.dto, 12
- com.usbcommander.dto.ConfigDTO, 17
 - allow, 19
 - allowedTime, 19
 - ConfigDTO, 18
 - frequency, 19
 - getAllowedTime, 18
 - getFrequency, 18
 - isAllow, 18
 - setAllow, 18
 - setAllowedTime, 18
 - setFrequency, 18
- com.usbcommander.dto.LogDTO, 30
 - code, 32
 - creationDate, 32
 - errorMessage, 32
 - getCode, 31
 - getCreationDate, 31
 - getErrorMessage, 31
 - getUsbList, 31

- getUsbValue, 31
- isUsbAllowed, 31
- LogDTO, 30
- setCode, 31
- setCreationDate, 31
- setErrorMessage, 31
- setUsbAllowed, 32
- setUsbList, 32
- setUsbValue, 32
- usbAllowed, 32
- usbList, 32
- usbValue, 32
- com.usbcommander.enums, 12
- com.usbcommander.enums.LogType, 35
 - code, 36
 - CONFIG_MOD, 36
 - CONNECTION, 36
 - ERROR, 36
 - getCode, 36
 - INCOHERENT, 36
 - INFO, 36
 - LogType, 36
 - MEMORY_CONN, 36
 - REGISTRY_MOD, 36
- com.usbcommander.errors, 12
- com.usbcommander.errors.NotConnectionException, 41
 - NotConnectionException, 41
- com.usbcommander.errors.ServiceDisabledException, 52
 - ServiceDisabledException, 52
- com.usbcommander.managers, 12
- com.usbcommander.managers.contract, 12
- com.usbcommander.managers.contract.IStatusManager, 25
 - deleteHistory, 26
 - generateLog, 26
 - getHistory, 27
 - instance, 27
 - IStatusManager, 26
- com.usbcommander.managers.contract.IUsbMemoryManager, 27
 - disableAccess, 28
 - enableAccess, 28
 - getAccessValue, 28
 - getConnectedDrives, 28
 - instance, 29
 - isDriveConnected, 29
 - IUsbMemoryManager, 28
 - removeExternalDrives, 29
- com.usbcommander.managers.StatusManager, 52
 - advapi32, 55
 - deleteHistory, 53
 - generateLog, 53, 54
 - getHistory, 54
 - getInstance, 54
 - machineConfig, 55
 - mapper, 55
 - StatusManager, 53
 - usbMemoryManager, 55
 - writeEventLog, 55
- com.usbcommander.managers.UsbMemoryManager, 58
 - disableAccess, 59
 - enableAccess, 59
 - getAccessValue, 59
 - getConnectedDrives, 59
 - getInstance, 60
 - IOCTL_STORAGE_EJECT_MEDIA, 61
 - isDriveConnected, 60
 - removeDrive, 60
 - removeExternalDrives, 60
 - UsbMemoryManager, 59
- com.usbcommander.services, 12
- com.usbcommander.services.contract, 13
- com.usbcommander.services.contract.CommanderService, 16
 - CommanderService, 16
 - run, 17
 - running, 17
 - start, 17
 - stopService, 17
- com.usbcommander.services.LogService, 33
 - getInstance, 34
 - instance, 34
 - LogService, 34
 - machineConfig, 34
 - mapper, 35
 - run, 34
 - sendLogs, 34
 - statusManager, 35
- com.usbcommander.services.SecurityService, 42
 - appConfigListener, 46
 - changeLogFrequency, 43
 - configMatchesMemory, 43
 - forceClose, 44
 - getInstance, 44
 - grantedAccess, 46
 - grantedFor, 46
 - instance, 46
 - machineConfig, 46
 - openAccess, 44
 - run, 45
 - SecurityService, 43
 - setListenerOn, 45
 - startListener, 45
 - statusManager, 46
 - usbMemoryManager, 47
 - usbStorListener, 47
 - usbStorMatchesMemory, 45
- com.usbcommander.services.ServerConnectionService, 47
 - CONN_ERROR, 50
 - connected, 50
 - connectToServer, 48
 - getInstance, 48

- instance, 50
- listenerThread, 49
- machineConfig, 50
- mapper, 50
- MAX_RETRY, 50
- MAX_RETRY_MSG, 51
- RECONNECT_TIMEOUT, 51
- run, 49
- SENDING_ERROR, 51
- sendMessage, 49
- SERVER_MSG_ERROR, 51
- ServerConnectionService, 48
- socket, 51
- statusManager, 51
- com.usbcommander.services.ThreadService, 56
 - getInstance, 57
 - instance, 57
 - machineConfig, 57
 - run, 57
 - statusManager, 57
 - ThreadService, 57
- CommanderService
 - com.usbcommander.services.contract.CommanderService, 16
- CONFIG_LOCATION
 - com.usbcommander.AppConst.ConfigReferences, 19
- CONFIG_MOD
 - com.usbcommander.enums.LogType, 36
- ConfigDTO
 - com.usbcommander.dto.ConfigDTO, 18
- configMatchesMemory
 - com.usbcommander.services.SecurityService, 43
- CONN_ERROR
 - com.usbcommander.services.ServerConnectionService, 50
- connected
 - com.usbcommander.services.ServerConnectionService, 50
- CONNECTION
 - com.usbcommander.enums.LogType, 36
- CONNECTION_ERROR_MESSAGE
 - com.usbcommander.AppConst.ErrorMessages, 20
- connectToServer
 - com.usbcommander.services.ServerConnectionService, 48
- creationDate
 - com.usbcommander.dto.LogDTO, 32
- deleteHistory
 - com.usbcommander.managers.contract.IStatusManager, 26
 - com.usbcommander.managers.StatusManager, 53
- DISABLE_VALUE
 - com.usbcommander.AppConst.RegistryReferences, 42
- disableAccess
 - com.usbcommander.managers.contract.IUsbMemoryManager, 28
- com.usbcommander.managers.UsbMemoryManager, 59
- ENABLE_VALUE
 - com.usbcommander.AppConst.RegistryReferences, 42
- enableAccess
 - com.usbcommander.managers.contract.IUsbMemoryManager, 28
 - com.usbcommander.managers.UsbMemoryManager, 59
- enableServerService
 - com.usbcommander.config.contract.IMachineConfig, 23
 - com.usbcommander.config.MachineConfig, 38
- ENTRY_NAME
 - com.usbcommander.AppConst.EventLogReferences, 21
- ENTRY_ROUTE
 - com.usbcommander.AppConst.EventLogReferences, 21
- ERROR
 - com.usbcommander.enums.LogType, 36
- ERROR_CODE
 - com.usbcommander.AppConst.EventLogReferences, 21
- ERROR_CONNECTION_CODE
 - com.usbcommander.AppConst.EventLogReferences, 21
- ERROR_DELETE
 - com.usbcommander.AppConst.ErrorMessages, 20
- errorMessage
 - com.usbcommander.dto.LogDTO, 32
- forceClose
 - com.usbcommander.services.SecurityService, 44
- frequency
 - com.usbcommander.dto.ConfigDTO, 19
- generateLog
 - com.usbcommander.managers.contract.IStatusManager, 26
 - com.usbcommander.managers.StatusManager, 53, 54
- getAccessValue
 - com.usbcommander.managers.contract.IUsbMemoryManager, 28
 - com.usbcommander.managers.UsbMemoryManager, 59
- getAllowedTime
 - com.usbcommander.dto.ConfigDTO, 18
- getCode
 - com.usbcommander.dto.LogDTO, 31
 - com.usbcommander.enums.LogType, 36
- getConnectedDrives
 - com.usbcommander.managers.contract.IUsbMemoryManager, 28
 - com.usbcommander.managers.UsbMemoryManager, 59

- getCreationDate
 - com.usbcommander.dto.LogDTO, 31
- getErrorMessage
 - com.usbcommander.dto.LogDTO, 31
- getFrequency
 - com.usbcommander.dto.ConfigDTO, 18
- getHistory
 - com.usbcommander.managers.contract.IStatusManager, 27
 - com.usbcommander.managers.StatusManager, 54
- getInstance
 - com.usbcommander.config.MachineConfig, 38
 - com.usbcommander.managers.StatusManager, 54
 - com.usbcommander.managers.UsbMemoryManager, 60
 - com.usbcommander.services.LogService, 34
 - com.usbcommander.services.SecurityService, 44
 - com.usbcommander.services.ServerConnectionService, 48
 - com.usbcommander.services.ThreadService, 57
- getIp
 - com.usbcommander.config.contract.IMachineConfig, 23
 - com.usbcommander.config.MachineConfig, 38
- getLogFrequency
 - com.usbcommander.config.contract.IMachineConfig, 23
 - com.usbcommander.config.MachineConfig, 39
- getPort
 - com.usbcommander.config.contract.IMachineConfig, 23
 - com.usbcommander.config.MachineConfig, 39
- getUsbList
 - com.usbcommander.dto.LogDTO, 31
- getUsbValue
 - com.usbcommander.dto.LogDTO, 31
- grantedAccess
 - com.usbcommander.services.SecurityService, 46
- grantedFor
 - com.usbcommander.services.SecurityService, 46
- INCOHERENT
 - com.usbcommander.enums.LogType, 36
- INCOHERENT_VALUE
 - com.usbcommander.AppConst.EventLogReferences, 21
- INFO
 - com.usbcommander.enums.LogType, 36
- INFO_CODE
 - com.usbcommander.AppConst.EventLogReferences, 22
- instance
 - com.usbcommander.config.contract.IMachineConfig, 25
 - com.usbcommander.managers.contract.IStatusManager, 27
 - com.usbcommander.managers.contract.IUsbMemoryManager, 29
 - com.usbcommander.services.LogService, 34
- com.usbcommander.services.SecurityService, 46
- com.usbcommander.services.ServerConnectionService, 50
- com.usbcommander.services.ThreadService, 57
- IOCTL_STORAGE_EJECT_MEDIA
 - com.usbcommander.managers.UsbMemoryManager, 61
- com.usbcommander.config.MachineConfig, 40
- IP_ENTRY
 - com.usbcommander.AppConst.ConfigReferences, 19
- isAllow
 - com.usbcommander.dto.ConfigDTO, 18
- isDriveConnected
 - com.usbcommander.managers.contract.IUsbMemoryManager, 29
 - com.usbcommander.managers.UsbMemoryManager, 60
- IStatusManager
 - com.usbcommander.managers.contract.IStatusManager, 26
- isUsbAllowed
 - com.usbcommander.dto.LogDTO, 31
- isUsbEnable
 - com.usbcommander.config.contract.IMachineConfig, 23
 - com.usbcommander.config.MachineConfig, 39
- IUsbMemoryManager
 - com.usbcommander.managers.contract.IUsbMemoryManager, 28
- JACKSON_ERROR_TEXT
 - com.usbcommander.AppConst.ErrorMessages, 20
- listenerThread
 - com.usbcommander.services.ServerConnectionService, 49
- LOG_ENTRY
 - com.usbcommander.AppConst.ConfigReferences, 19
- LOG_GENERATION_ERROR
 - com.usbcommander.AppConst.ErrorMessages, 20
- LogDTO
 - com.usbcommander.dto.LogDTO, 30
- logFrequency
 - com.usbcommander.config.MachineConfig, 40
- LogService
 - com.usbcommander.services.LogService, 34
- LogType
 - com.usbcommander.enums.LogType, 36
- MachineConfig
 - com.usbcommander.config.MachineConfig, 38
- machineConfig
 - com.usbcommander.managers.StatusManager, 55
 - com.usbcommander.services.LogService, 34
 - com.usbcommander.services.SecurityService, 46

- com.usbcommander.services.ServerConnectionService, 50
- com.usbcommander.services.ThreadService, 57
- main
 - com.usbcommander.App, 15
- MAIN_LOCATION
 - com.usbcommander.AppConst, 15
- mapper
 - com.usbcommander.managers.StatusManager, 55
 - com.usbcommander.services.LogService, 35
 - com.usbcommander.services.ServerConnectionService, 50
- MAX_RETRY
 - com.usbcommander.services.ServerConnectionService, 50
- MAX_RETRY_MSG
 - com.usbcommander.services.ServerConnectionService, 51
- MEMORY_CONN
 - com.usbcommander.enums.LogType, 36
- MEMORY_CONNECTED
 - com.usbcommander.AppConst.EventLogReferences, 22
- MIN_FRECUENNCY
 - com.usbcommander.AppConst.ConfigReferences, 19
- NotConnectionException
 - com.usbcommander.errors.NotConnectionException, 41
- openAccess
 - com.usbcommander.services.SecurityService, 44
- port
 - com.usbcommander.config.MachineConfig, 41
- PORT_ENTRY
 - com.usbcommander.AppConst.ConfigReferences, 19
- RECONNECT_TIMEOUT
 - com.usbcommander.services.ServerConnectionService, 51
- REGISTRY_MOD
 - com.usbcommander.enums.LogType, 36
- REGISTRY_MODIFICATION
 - com.usbcommander.AppConst.EventLogReferences, 22
- removeDrive
 - com.usbcommander.managers.UsbMemoryManager, 60
- removeExternalDrives
 - com.usbcommander.managers.contract.IUsbMemoryManager, 29
 - com.usbcommander.managers.UsbMemoryManager, 60
- run
 - com.usbcommander.services.contract.CommanderService, 17
- com.usbcommander.services.LogService, 34
- com.usbcommander.services.SecurityService, 45
- com.usbcommander.services.ServerConnectionService, 49
- com.usbcommander.services.ThreadService, 57
- running
 - com.usbcommander.services.contract.CommanderService, 17
- saveConfig
 - com.usbcommander.config.contract.IMachineConfig, 24
 - com.usbcommander.config.MachineConfig, 39
- SecurityService
 - com.usbcommander.services.SecurityService, 43
- SENDING_ERROR
 - com.usbcommander.services.ServerConnectionService, 51
- sendLogs
 - com.usbcommander.services.LogService, 34
- sendMessage
 - com.usbcommander.services.ServerConnectionService, 49
- SERVER_MSG_ERROR
 - com.usbcommander.services.ServerConnectionService, 51
- ServerConnectionService
 - com.usbcommander.services.ServerConnectionService, 48
- SERVICE_NOT_RUNNING
 - com.usbcommander.AppConst.ErrorMessages, 20
- ServiceDisabledException
 - com.usbcommander.errors.ServiceDisabledException, 52
- setAllow
 - com.usbcommander.dto.ConfigDTO, 18
- setAllowedTime
 - com.usbcommander.dto.ConfigDTO, 18
- setCode
 - com.usbcommander.dto.LogDTO, 31
- setCreationDate
 - com.usbcommander.dto.LogDTO, 31
- setErrorMessage
 - com.usbcommander.dto.LogDTO, 31
- setFrequency
 - com.usbcommander.dto.ConfigDTO, 18
- setListenerOn
 - com.usbcommander.services.SecurityService, 45
- setLogFrequency
 - com.usbcommander.config.contract.IMachineConfig, 24
 - com.usbcommander.config.MachineConfig, 40
- setUsbAllowed
 - com.usbcommander.dto.LogDTO, 32
- setUsbEnable
 - com.usbcommander.config.contract.IMachineConfig, 24
 - com.usbcommander.config.MachineConfig, 40
- setUsbList

- com.usbcommander.dto.LogDTO, 32
- setUsbValue
 - com.usbcommander.dto.LogDTO, 32
- shouldAnswerWithTrue
 - com.usbcommander.AppTest, 16
- socket
 - com.usbcommander.services.ServerConnectionService, 51
- src Directory Reference, 9
- src/main Directory Reference, 8
- src/main/java Directory Reference, 8
- src/main/java/com Directory Reference, 7
- src/main/java/com/usbcommander Directory Reference, 9
- src/main/java/com/usbcommander/config Directory Reference, 7
- src/main/java/com/usbcommander/config/contract Directory Reference, 8
- src/main/java/com/usbcommander/dto Directory Reference, 8
- src/main/java/com/usbcommander/enums Directory Reference, 8
- src/main/java/com/usbcommander/errors Directory Reference, 8
- src/main/java/com/usbcommander/managers Directory Reference, 8
- src/main/java/com/usbcommander/managers/contract Directory Reference, 8
- src/main/java/com/usbcommander/services Directory Reference, 9
- src/main/java/com/usbcommander/services/contract Directory Reference, 8
- src/test Directory Reference, 9
- src/test/java Directory Reference, 8
- src/test/java/com Directory Reference, 7
- src/test/java/com/usbcommander Directory Reference, 9
- start
 - com.usbcommander.services.contract.CommanderService, 17
- startListener
 - com.usbcommander.services.SecurityService, 45
- StatusManager
 - com.usbcommander.managers.StatusManager, 53
- statusManager
 - com.usbcommander.services.LogService, 35
 - com.usbcommander.services.SecurityService, 46
 - com.usbcommander.services.ServerConnectionService, 51
 - com.usbcommander.services.ThreadService, 57
- stopService
 - com.usbcommander.services.contract.CommanderService, 17
- ThreadService
 - com.usbcommander.services.ThreadService, 57
- UNAUTHORIZED_CONFIGURATION_MODIFICATION
 - com.usbcommander.AppConst.EventLogReferences, 22
- USB_ALLOW_ENTRY
 - com.usbcommander.AppConst.ConfigReferences, 20
- USB_ENUM
 - com.usbcommander.AppConst.RegistryReferences, 42
- USB_STOR
 - com.usbcommander.AppConst.RegistryReferences, 42
- usbAllowed
 - com.usbcommander.dto.LogDTO, 32
- usbEnable
 - com.usbcommander.config.MachineConfig, 41
- usbList
 - com.usbcommander.dto.LogDTO, 32
- UsbMemoryManager
 - com.usbcommander.managers.UsbMemoryManager, 59
- usbMemoryManager
 - com.usbcommander.managers.StatusManager, 55
 - com.usbcommander.services.SecurityService, 47
- usbStorListener
 - com.usbcommander.services.SecurityService, 47
- usbStorMatchesMemory
 - com.usbcommander.services.SecurityService, 45
- usbValue
 - com.usbcommander.dto.LogDTO, 32
- WEIRDVALUE_ERROR
 - com.usbcommander.AppConst.ErrorMessages, 21
- writeEventLog
 - com.usbcommander.managers.StatusManager, 55